

Configuración:

Settings

SaveDone



 **Providers**



Configuration Profile



Root Code

+

Save different API configurations to quickly switch between providers and settings.



API Provider [Roo Code Cloud documentation](#)



Roo Code Cloud

Securely authenticated through your Roo Code Cloud account.



Model



roo/code-supernova-1-million

A versatile agentic coding stealth model with a 1M token context window that supports image inputs, accessible for free through Roo Code Cloud for a limited time. (Note: the free prompts and completions are logged by the model ... [More](#)



Context Window: 1,000,000 tokens

Max output: 30,000 tokens

✓ Supports images

✗ Does not support computer use

✓ Supports prompt caching



> **Advanced settings**



Registro de prompts:

Paso	Prompt Exacto
1.1 – Diseño Esquemático y Rutas	Actúa como un arquitecto de software. Diseña un sistema RESTful para gestionar citas médicas simples. La entidad principal es Appointment con los campos: id (UUID), client_name (string), date (date), time (string en formato HH:MM), y service_type (string). Proporciona: - El esquema de la tabla en PostgreSQL (con tipos de datos y restricciones). - Las rutas API RESTful (CRUD) necesarias, indicando método HTTP, endpoint y propósito.
1.2 – Estrategia de Validación	Como arquitecto backend, diseña una estrategia para evitar la creación de citas duplicadas en el mismo date y time. Describe en pseudocódigo o pasos lógicos cómo implementarías esta validación dentro de la función POST /appointments antes de insertar en la base de datos. Asume que usas Node.js, Express y una base de datos PostgreSQL (como Supabase).
1.3 – Plan de Desarrollo Frontend	Como arquitecto frontend, proporciona un plan paso a paso para construir una interfaz mínima de reserva de citas usando HTML, CSS y JavaScript vanilla. El formulario debe incluir campos para: nombre del cliente, fecha, hora (en formato HH:MM) y tipo de servicio. Al enviar, debe hacer una petición POST a /appointments. Describe cómo estructurar el HTML, validar mínimamente en el frontend y enviar los datos al backend.
2.1 – Generación de la función POST con validación	Actúa como desarrollador backend en Node.js con Express. Escribe la función completa para la ruta POST /appointments que: - Recibe client_name, date (como string YYYY-MM-DD), time (como string HH:MM), y service_type. - Antes de insertar, consulta la base de datos (usando Supabase client) para verificar si ya existe una cita con la misma date y time. - Si existe, responde con status 409 y mensaje "Cita ya reservada". - Si no existe, inserta la nueva cita y responde con status 201 y los datos guardados. Usa async/await y manejo básico de errores.
2.2 – Refactorización: estandarizar formatos	Modifica la función POST /appointments del paso anterior para asegurar que: - El campo date se almacene como una cadena en formato ISO 8601 (YYYY-MM-DD). - El campo time se almacene como una cadena en formato HH:MM (24h). Valida y normaliza estos campos al recibirlos (por ejemplo, si llegan con milisegundos o AM/PM, corrígelos o recházalos). Si el formato es inválido, responde con status 400.
2.3 – Identificación y corrección de un bug	Revisa la siguiente función POST para crear citas (incluye el código que generaste en el paso 2.2). He introducido intencionalmente un error lógico en la validación de duplicados: en la comparación de date y time, se usa == en lugar de verificar correctamente si hay resultados. Analiza el código, identifica el bug y corrige la lógica para que rechace correctamente las citas duplicadas.

Análisis de Uso:

El Modo Architect permite pensar estratégicamente: genera diseños de alto nivel, esquemas y planes sin ataduras de sintaxis, ideal para definir estructura y lógica antes de codificar.

El Modo Code/Developer, en cambio, se enfoca en la implementación concreta: produce código funcional, detallado y listo para ejecutar, respetando el contexto previo.

Usar ambos en secuencia mejora la calidad del resultado: primero se diseña bien, luego se codifica con precisión.

Código Final:

Server.js:

```
require('dotenv').config();

const express = require('express');
```

```
const appointmentsRouter = require('./routes/appointments');
```

```
const app = express();
```

```
const port = process.env.PORT || 3000;
```

```
app.use(express.json());
```

```
app.use(express.static('public'));
```

```
app.use('/appointments', appointmentsRouter);
```

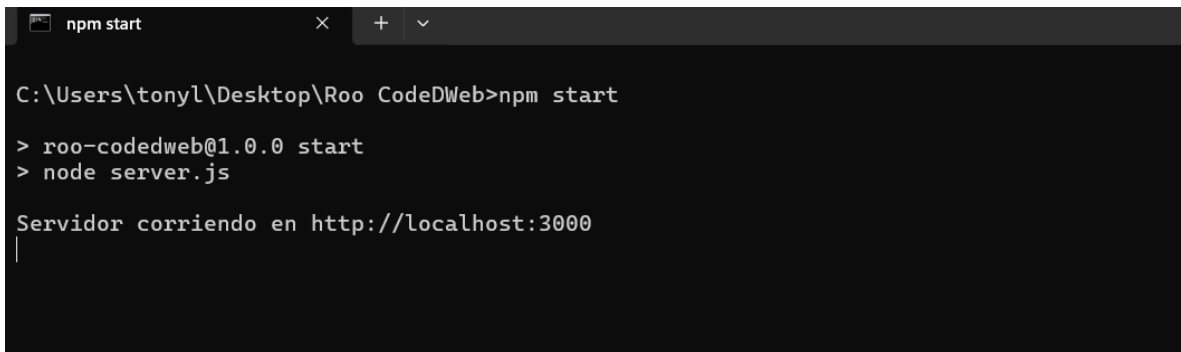
```
app.listen(port, () => {
```

```
  console.log(`Servidor corriendo en http://localhost:${port}`);
```

```
});
```

Capturas:

Ejecución del Servidor:



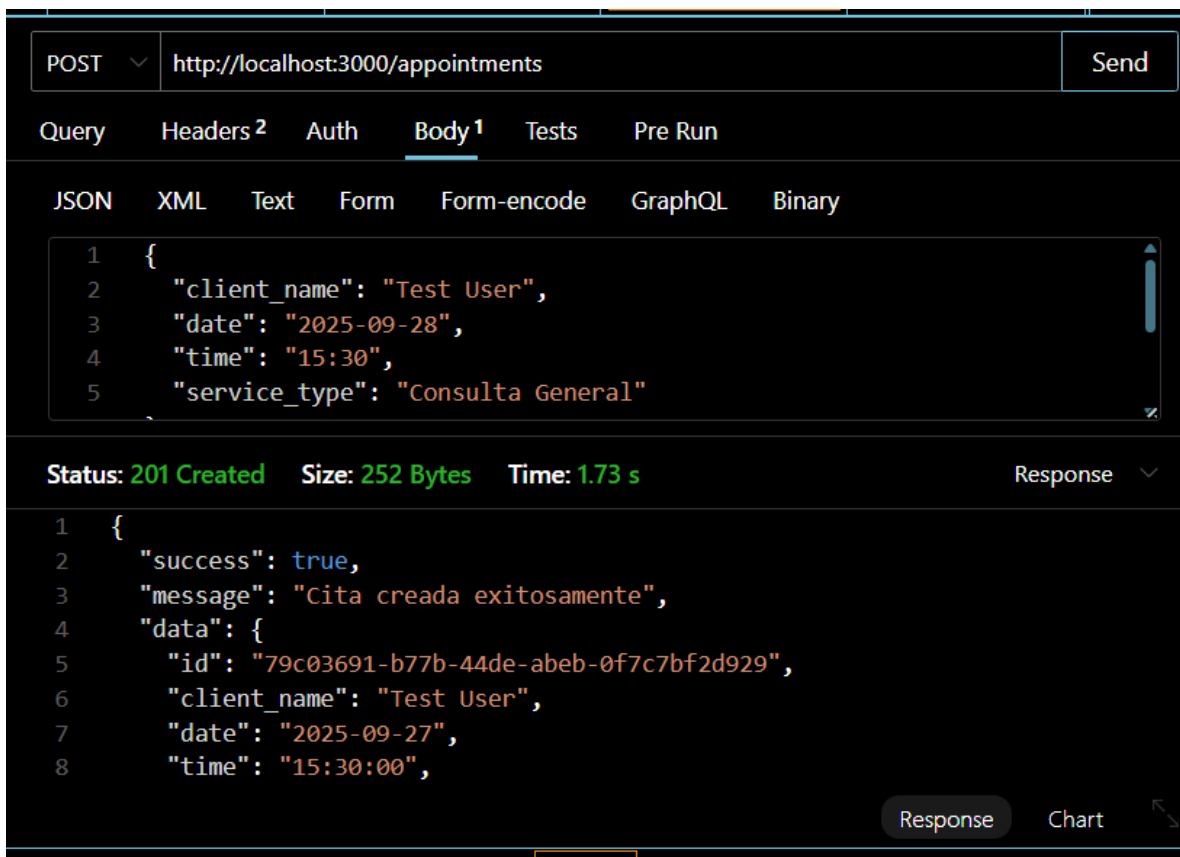
```
npm start

C:\Users\tonyl\Desktop\Roo CodeDWeb>npm start

> roo-codedweb@1.0.0 start
> node server.js

Servidor corriendo en http://localhost:3000
|
```

Funcionalidad:



Frontend:

